

Model Conversion

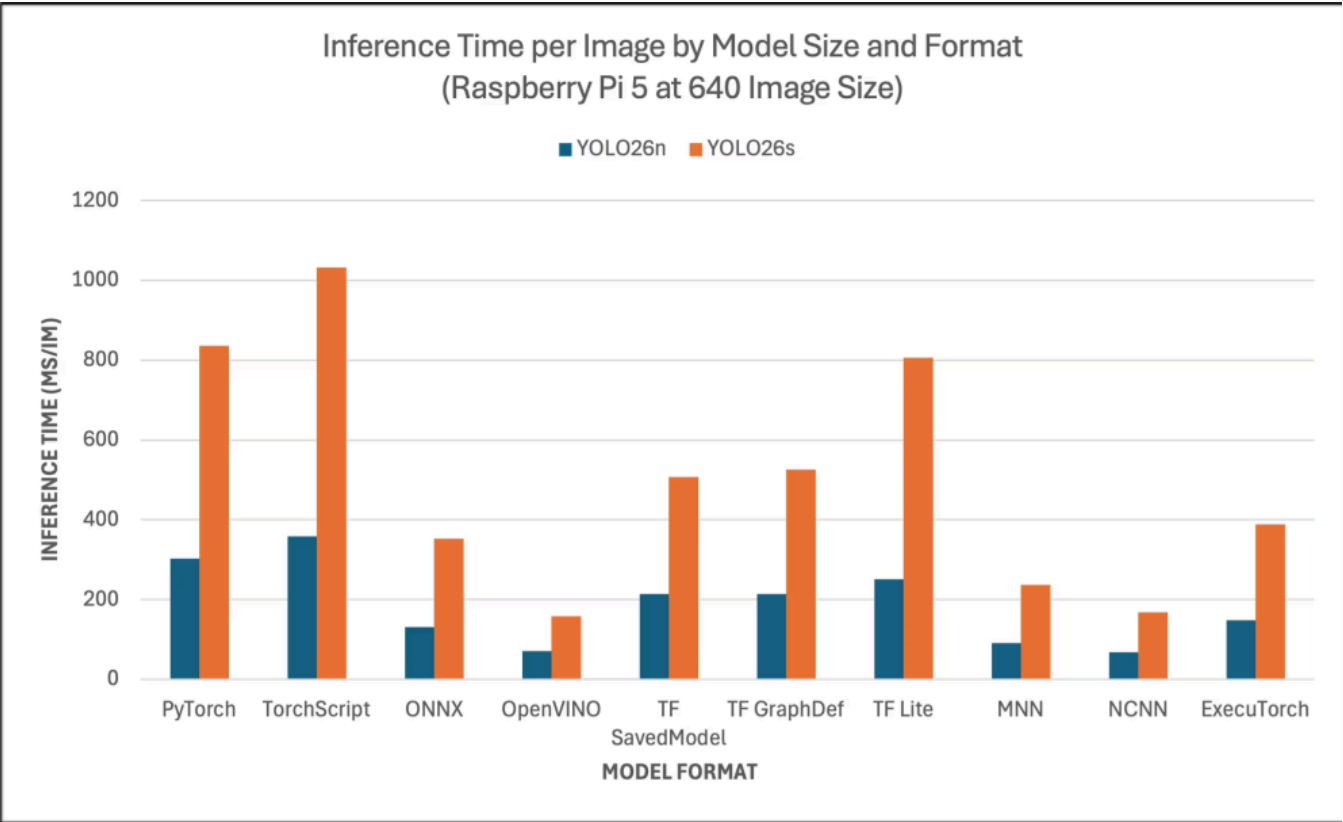
Model Conversion

- 1、YOLO26 (Benchmark)
 - Comparison Chart
- 2、Model Conversion
 - 2.1、CLI: pt → onnx, pt → ncnn
 - 2.2、Python: pt → onnx → ncnn
- 4、Model Prediction
 - CLI Usage
- References

1、YOLO26 (Benchmark)

Ultralytics team benchmarked YOLO26 on ten different model formats, measuring speed and accuracy, including: PyTorch, TorchScript, ONNX, OpenVINO, TF SavedModel, TF GraphDef, TF Lite, MNN, NCNN, and ExecuTorch. The benchmark tests were run on Raspberry Pi 5 at FP32 precision with a default input image size of 640.

Comparison Chart



2、Model Conversion

The first time you use the export mode of YOLO26, some dependencies will be automatically installed. Please wait for it to complete automatically!

2.1、CLI: pt → onnx, pt → ncnn

Convert PyTorch format models to onnx and ncnn

```
cd /home/sunrise/ultralalytics26
```

```
yolo export model=yolo26n.pt format=onnx
# yolo export model=yolo26n-seg.pt format=onnx
# yolo export model=yolo26n-pose.pt format=onnx
# yolo export model=yolo26n-cls.pt format=onnx
# yolo export model=yolo26n-obb.pt format=onnx

yolo export model=yolo26n.pt format=ncnn
# yolo export model=yolo26n-seg.pt format=ncnn
# yolo export model=yolo26n-pose.pt format=ncnn
# yolo export model=yolo26n-cls.pt format=ncnn
# yolo export model=yolo26n-obb.pt format=ncnn
```

```
ONNX: starting export with onnx 1.20.1 opset 20...
/home/sunrise/.local/lib/python3.10/site-packages/torch/onnx/_internal/torchscript_exporter/symbolic_opset11.py:954: UserWarning: Exporting aten::index operator of advanced indexing in opset 20 is achieved by combination of multiple ONNX operators, including Reshape, Transpose, Concat, and Gather. If indices include negative values, the exported graph will produce incorrect results.
  return opset9.index(g, self, index)
ONNX: slimming with onnxslim 0.1.90...
2026-03-26 14:11:34.369281992 [W:onnxruntime:Default, device_discovery.cc:164 DiscoverDevicesForPlatform] GPU device discovery failed: device_discovery.cc:89 ReadFileContents Failed to open file: "/sys/class/drm/card0/device/vendor"
ONNX: export success ✓ 51.5s, saved as 'yolo26n.onnx' (9.5 MB)

Export complete (55.9s)
Results saved to /home/sunrise/ultralalytics26
Predict:      yolo predict task=detect model=yolo26n.onnx imgsz=640
Validate:     yolo val task=detect model=yolo26n.onnx imgsz=640 data=/home/lq
/codes/ultralalytics/ultralalytics/cfg/datasets/coco.yaml
Visualize:    https://netron.app
[01F] Learn more at https://docs.ultralalytics.com/modes/export
```

```

inline module = ultralytics.nn.modules.block.SPPF
inline module = ultralytics.nn.modules.conv.Concat
inline module = ultralytics.nn.modules.conv.Conv
inline module = ultralytics.nn.modules.conv.DWConv
inline module = ultralytics.nn.modules.head.Detect

-----

##### pass_level1
##### pass_level2
##### pass_level3
##### pass_level4
##### pass_level5
##### pass_ncnn
NCNN: export success ✓ 93.2s, saved as 'yolo26n_ncnn_model' (9.4 MB)

Export complete (97.5s)
Results saved to /home/sunrise/ultralytics26
Predict:      yolo predict task=detect model=yolo26n_ncnn_model imgsz=640
Validate:     yolo val task=detect model=yolo26n_ncnn_model imgsz=640 data=/h
ome/lq/codes/ultralytics/ultralytics/cfg/datasets/coco.yaml
Visualize:    https://netron.app
📄 Learn more at https://docs.ultralytics.com/modes/export

```

2.2、Python: pt → onnx → ncnn

Convert PyTorch format models to TensorRT: The conversion process will automatically generate the ONNX model

```
cd /home/sunrise/ultralytics26/yahboom_demo
```

```
python3 model_pt_onnx_ncnn.py
```

```

from ultralytics import YOLO

# Load a YOLO PyTorch model
model = YOLO("/home/sunrise/ultralytics26/yolo26n.pt")
# model = YOLO("/home/sunrise/ultralytics26/yolo26n-seg.pt")
# model = YOLO("/home/sunrise/ultralytics26/yolo26n-pose.pt")
# model = YOLO("/home/sunrise/ultralytics26/yolo26n-cls.pt")
# model = YOLO("/home/sunrise/ultralytics26/yolo26n-obb.pt")

# Export the model to ONNX format
model.export(format="onnx") # This will create 'yolo11n.onnx' in the same directory

# Export the model to NCNN format
model.export(format="ncnn") # creates 'yolo11n_ncnn_model'

```

Note: The converted model files are located at the converted model file location

```

inline module = ultralytics.nn.modules.conv.Conv
inline module = ultralytics.nn.modules.conv.DWConv
inline module = ultralytics.nn.modules.head.Detect

-----

##### pass_level1
##### pass_level2
##### pass_level3
##### pass_level4
##### pass_level5
##### pass_ncnn
NCNN: export success ✔ 27.7s, saved as '/home/sunrise/ultralytics26/yolo26n_ncnn_model' (9.4 MB)

Export complete (32.7s)
Results saved to /home/sunrise/ultralytics26
Predict:      yolo predict task=detect model=/home/sunrise/ultralytics26/yolo26n_ncnn_model imgsz=640
Validate:     yolo val task=detect model=/home/sunrise/ultralytics26/yolo26n_ncnn_model imgsz=640 data=/home/lq/codes/ultralytics/ultralytics/cfg/datasets/coco.yaml
Visualize:    https://netron.app
sunrise@ubuntu:~/ultralytics26/yahboom_demo$

```

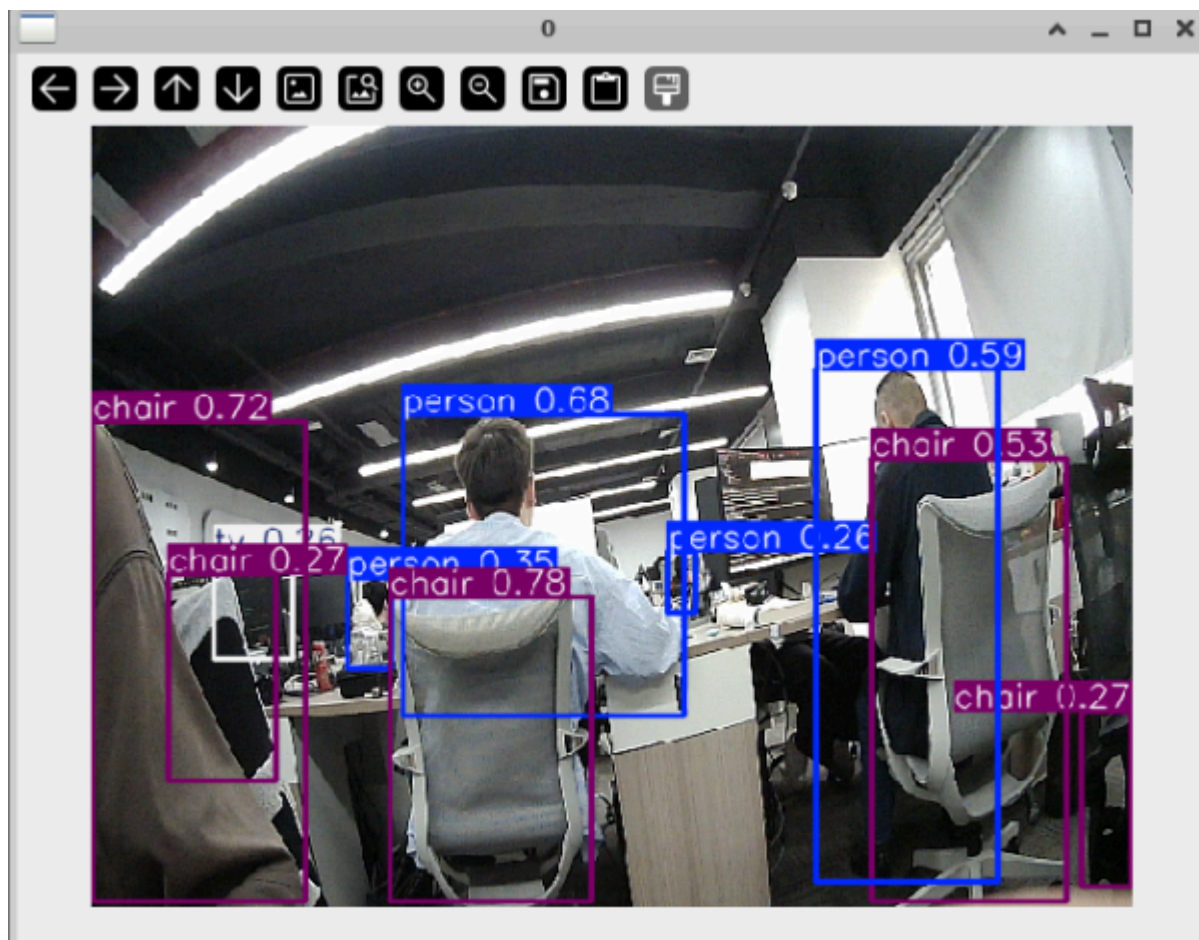
4、 Model Prediction

CLI Usage

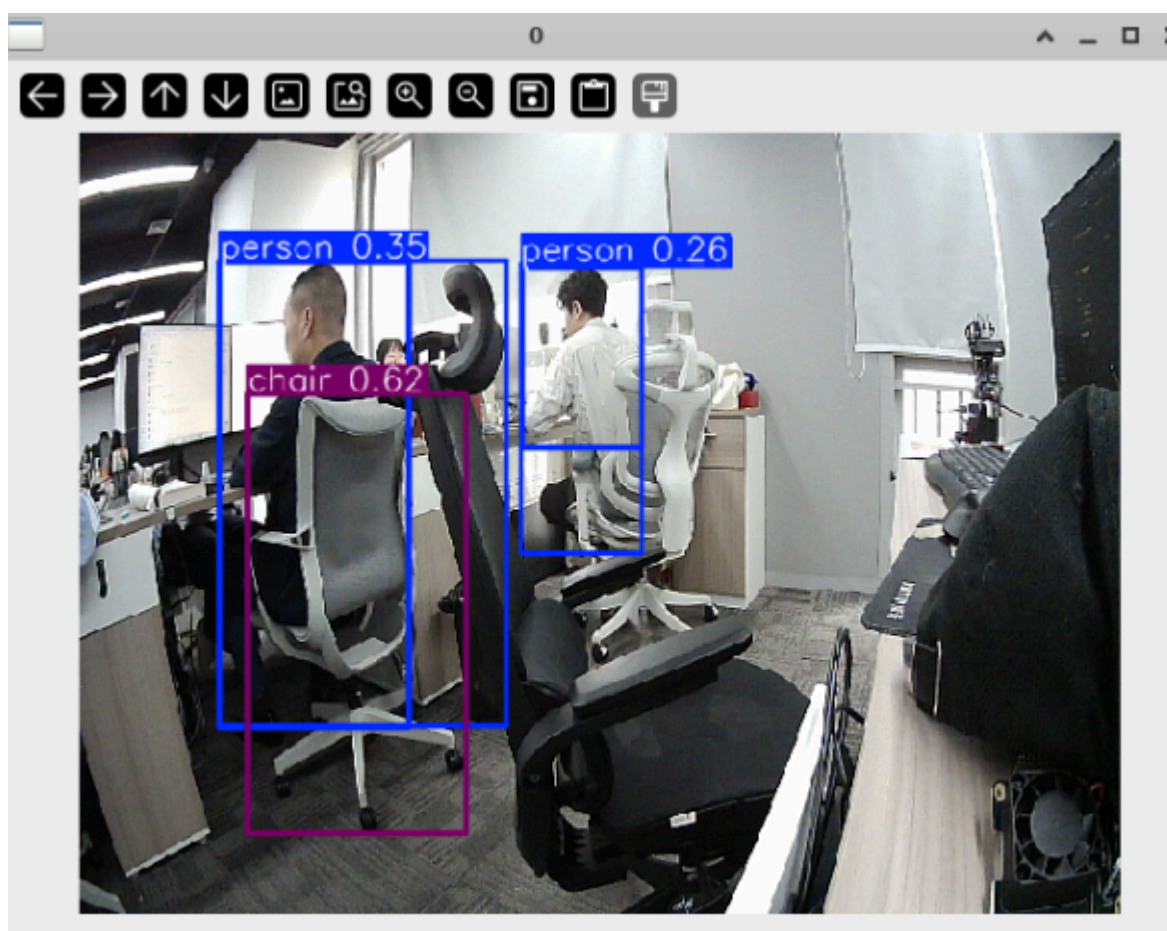
CLI currently only supports USB camera. CSI camera users can directly modify the previous Python code to call onnx and ncnn models!

```
cd /home/sunrise/ultralytics26
```

```
yolo predict model=yolo26n.onnx source=0 save=False show
```



```
yolo predict model=yolo26n_ncnn_model source=0 save=False show
```



References

[Quick Start Guide: Raspberry Pi with Ultralytics YOLO26](#)